AI-Native Engineer Capstone

LearnLab — Lightweight Learning Management System

Business Case BC-AINE-011 Domain: EdTech Online Learning

1. Project Identity

Business Case Title	Lightweight Learning Management System
Business Case ID	BC-AINE-011
Domain	EdTech Online Learning
Project Type	AI-Native Engineering Capstone
Cohort	(filled by Operations team)

2. Engagement Overview

Duration	20 hours
Format	Individual
Evaluation Mode	Automated review of submitted Git repository against the AI-Native Engineering Rubric (21 parameters / 100 marks across 5 categories)
Submission	Push final code to your assigned Virtusa GitLab repository by the cohort cut-off
Review Output	Per-learner Excel report (Summary, Categories, Scorecard, Detailed, Improvement)
Grade Bands	Distinction ≥ 90 Merit 75-89 Pass 60-74 Not Yet Passed < 60

What is evaluated: AI-native engineering practices (specs, agents, skills, hooks, CLAUDE.md hierarchy, plugin packaging, MCP, Harness plugin usage), business understanding (context, rules, AC), architectural discipline (structural rules, TDD), technical implementation (code, tests, coverage, UI validation), and CI/CD integration (PR-driven workflow, Claude in CI).

What is not evaluated: the specific technology stack chosen, the visual polish of the UI, or the use of any AI provider beyond Claude Code.

3. Problem Statement & Expected Solution

3.1 Problem

An online learning platform wants to launch a lightweight LMS-course catalog, lesson delivery. enrollments, progress tracking, and quizzes under one constraint: no manual coding. All external integrations (video streaming, payments) are stubbed. Every line of production code must be generated by Claude Code agents under your supervision.

3.2 Your Role

AI-Native Engineer architect, supervisor, and quality gatekeeper. You write specifications, configure agents, install guardrails, and review pull requests. You do not write production code by hand.

3.3 Expected Solution

A working platform delivered as a Git repository that demonstrates:

- A complete AI-native engineering substrate built on the Claude Harness Engine plugin foundation, extended with learning-management-specific specs, agents, skills, hooks, commands, and MCP integration.

- A functional LMS built end-to-end by Claude Code agents covering course authoring, learner enrollment, progress tracking, quiz auto-grading, and instructor/admin workflows with no hand-written production code.

- An evidence trail (PRs, agent logs, post-mortems, sprint contracts) proving the work was agent-led.

3.4 Applicable Rules

- **No-Hand-Coding Rule**. All production code, tests, and migrations must be generated by Claude Code agents. Hand-edits are limited to specs, CLAUDE.md, agent definitions, hooks, skills, and other substrate files.

- **Spec-Is-Truth Rule.** Whenever a spec and code disagree, the spec wins. Update the spec deliberately and regenerate the code.

- **PR-Only Merge Rul**e. No direct commits to main. All changes flow through agent-reviewed pull requests.
- **Synthetic-Data Rule.** Use only dummy data. Do not use Virtusa confidential or customer-sensitive data for this project.

4. Technology Stack

Choose any stack that matches your background. The stack does not affect evaluation.

Layer	Options
Frontend	React, Angular, Vue, server-rendered (Thymeleaf / Razor)
Backend	Java + Spring Boot, .NET ASP.NET Core Web API, Python + FastAPI, Node.js + Express / NestJS
Database	In-memory (H2/SQLite), RDBMS (MySQL/MariaDB / SQL Server), NoSQL (MongoDB / DynamoDB)
Build & Dependency Management	Maven/Gradle, dotnet CLI, pip, Poetry, npm/pnpm
Testing	JUnit/NUnit/pytest/Vitest, Playwright (E2E), ArchUnit/dependency-cruiser/import-linter
CI	GitLab CI (preferred for Virtusa), GitHub Actions, with Claude Code Action integrated
MCP Server	Playwright MCP
Required Plugin	Claude Harness Engine plugin (Generator-Evaluator harness for autonomous long-running development)

5. Acceptance Criteria & Non-Functional Requirements

5.1 Functional Acceptance Criteria

ID	Criterion
AC-01	Learner can register and enroll in any PUBLISHED course; a learner cannot enroll twice in the same course
AC-02	Instructor can create a course with title, description, category, and difficulty; courses start in DRAFT state
AC-03	Instructor can add modules and nested lessons to a draft course (course modules lessons hierarchy)
AC-04	Admin can publish (DRAFT PUBLISHED) or unpublish (PUBLISHED→ UNPUBLISHED) a course; learners can only enroll in PUBLISHED courses
AC-05	Quiz supports MCQ (single correct) and TRUE/FALSE question types; quizzes are auto-graded by a deterministic rule
AC-06	Learner can attempt a quiz multiple times, best score is recorded; each attempt is timestamped and append-only
AC-07	Learner can mark a lesson complete; course progress percentage is computed as completed_lessons/total_lessons
AC-08	Course lifecycle states DRAFT PUBLISHED UNPUBLISHED ARCHIVED are enforced; invalid transitions raise InvalidCourseStateException
AC-09	Instructor can view learner progress aggregated by course (count by progress band: 0-25%, 25-50%, 50-75%, 75-100%)
AC-10	Admin can manage course catalog, approve published courses, and monitor enrollment-level analytics

5.2 Non-Functional Requirements

ID	**Requirement**
NFR-01	Quiz scores are computed deterministically in integer percentage; calculations never use floating-point
NFR-02	Quiz attempts, lesson completions, and enrollment records are append-only

ID	Requirement
NFR-03	Learner PII (email, phone, performance data) is never written to logs in plaintext
NFR-04	Authentication boundary at controller layer; leaner/instructor/admin roles are separated
NFR-05	Database migrations are append-only
NFR-06	Structured JSON logs with request correlation IDs
NFR-07	Health endpoint returns 200 within 1 second of a successful startup
NFR-08	Architecture rules enforced as tests - instructors cannot grade their own courses' learners; a learner cannot access another learner's progress; published courses cannot be mutated without an unpublish transition

6. Functional Scope

6.1 Customer / End-User Functionality (User Journeys)

- **Browse.** Search published courses by category and difficulty.
- **Enroll**. Enroll in a course; one enrollment per learner-course pair.
- **Learn**. Move through modules and lessons sequentially; mark lessons complete.
- **Assess**. Attempt module-end quizzes; multiple attempts allowed; best score retained.
- **Progress**. View own progress percentage and quiz scores across enrolled courses.

6.2 Internal & System Functionality

- **Instructor Workbench.** Author courses with modules, lessons, and quiz questions; view learner progress aggregated by course.

- **Admin Console.** Manage course catalog, publish / unpublish courses, monitor enrolment and engagement. I

- **Quiz Engine.** Auto-grade MCQ and TRUE / FALSE attempts deterministically; persist attempt history.

- **Progress Tracker.** Compute and snapshot course progress for each learner.

6.3 Out of Scope

You do not need to build:

- Real video streaming or CDN integration

- Real payment gateway/paid courses

- Live virtual classes (WebRTC)

- Real proctoring/anti-cheat technology

- Certification generation and external verification

- Production deployment, secret management, multi-region

7. AI-Native Implementation Expectations

Foundation. Install the Claude Harness Engine plugin first. The harness ships 7 base agents (planner, generator, evaluator, security-reviewer, test-engineer, design-critic, ui-designer), 12+ hooks, base commands, sprint contracts, an evaluator wired to Playwright MCP, and a self-healing loop. On top of that foundation, your repository must add the project-specific substrate listed in this section. The rubric scores the final repository-both Harness-supplied and learner-added artifacts contribute, but the thresholds below are calibrated so that learners who only install Harness without customizing for the project domain will not meet them.

7.1 Claude Code Specs (40 marks)

- specs/app_spec.md (root spec) and specs/<feature>_spec.md per major feature, each with explicit Acceptance Criteria sections

- Layered CLAUDE.md root + per-module + per-folder; AGENTS.md as a table of contents only

- At least 2 project-specific skills under claude/skills/ added by you (examples: quiz-grader, progress-aggregator, course-state-validator, enrollment-deduper, or technical: spec-to-test-generator, archtest-author, crud-scaffolder)

- At least 2 project-specific custom commands under claude/commands/added by you (beyond Harness defaults)

- At least 2 project-specific agents under claude/agents/ added by you. These may be **domain agents** (e.g., quiz-grader-agent, progress-tracker-agent, course-publisher-agent,enrollment-manager-agent), **technical agents** that strengthen the engineering substrate (e.g., state-machine-validator-agent, archtest-author-agent, crud-scaffolder-agent), or a mix of both. You may rename Harness's generator →implementer, evaluator →reviewer, test-engineer →tester to align with canonical naming

- At least one programmatic Claude Agent SDK usage in scripts/

- Your own plugin.json packaging your project's substrate (in addition to Harness's plugin.json)

- At least one project-specific debugging-log or post-mortem document under docs/ evidencing environment-first resolution

- Playwright MCP configured in .mcp.json and actively used by the evaluator

7.2 Business Understanding (20 marks)

- docs/business-case.md covering: problem, target users, success metrics, domain rules, value proposition. This is the document the reviewer plugin reads - see Section 8

- At least 4 rule/policy/ validator files under src/domain/ (or equivalent) implementing learning-management business rules

- Frontend evidence (any of: React/Angular / Vue) with at least one responsive layout

- Acceptance criteria expressed in testable form - Given-When-Then or AC-NN identifiers - for at least 4 specs

7.3 Architectural Discipline (8 marks)

- Architecture-test directory (tests/architecture/ or equivalent) with at least 3 structural tests

- ArchUnit/dependency-cruiser/import-linter configured to enforce layering rules

- TDD discipline document at docs/tdd.md and at least 10 test files committed across history showing red-green-refactor pattern

7.4 Technical Implementation (26 marks)

- docs/architecture.md with at least 1 architecture diagram (Mermaid or image) and an explicit layered structure (controllers/services/repositories/domain)
- Substantive codebase with at least 3000 lines of meaningful generated code
- Playwright (or equivalent) for UI validation with at least one snapshot directory
- Tests tagged with AC IDs every spec AC has at least 1 corresponding test
- At least 20 unit tests with a coverage artefact (coverage.xml, lcov.info, or equivalent) and the coverage tool declared in dependencies

7.5 CI/CD (6 marks)

- gitlab-ci.yml (or .github/workflows/) with the build and test pipeline
- Claude Code Action (or equivalent) wired into the CI workflow
- At least 3 PR-driven merges (use git merge --no-ff to preserve PR history); zero direct commits to main

8. Expected Outcomes & Deliverables

By the end of 20 hours, the submitted Git repository must contain the Mandatory items below. Good-to-Have items differentiate Merit and Distinction submissions.

8.1 Mandatory

1. **Working Application.** LMS deployable locally via a single command (./mvnw spring-boot:run, npm start, dotnet run, or equivalent) with seed data (instructors, courses, learners, sample quizzes) and a README quick-start.
2. **docs/business-case.md.** The business context document the reviewer plugin reads. Use this brief's Sections 3 and 6 as the source; rewrite in your own words covering problem, users, success metrics, domain rules, and value proposition.
3. **Specifications**. specs/app_spec.md plus per-feature specs (course-authoring, catalog, enrollment, quiz-engine, progress-tracker, instructor-workbench) with explicit Acceptance Criteria sections.

4. **Layered CLAUDE.md Hierarchy**. Root + per-module + per-folder, AGENTS.md as a table of contents.

5. **Project Agent Substrate.**.claude/agents/, claude/skills/, claude/commands/, claude/hooks/Harness foundation plus your project-specific additions as listed in Section 7.1.

6. **Claude Harness Engine Plugin Integration.** Plugin installed; project scaffolded via the harness; at least one full sprint cycle (Generator Evaluator ratcheting PR) driven through it. Sprint contracts and evaluator outputs committed under sprint-contracts/ and specs/reviews/.

7. **Plugin Manifest**. Your own plugin.json at the repository root packaging your learning-management substrate; mcp.json with Playwright MCP active.

8. **Test Suite.** Unit tests (incl. quiz auto-grading, progress aggregation, role-boundary checks, state transitions), architecture tests, AC-tagged tests, Playwright UI tests, coverage report - all committed.

9. **CI Pipeline.** gitlab-ci.yml (or equivalent) with Claude Code Action wired in; passing build on main.

10. **PR-Driven Git History**. At least 3 PR-driven merges; zero direct pushes to main.

8.2 Good-to-Have

11. Architecture & TDD Documentation, docs/architecture.md with at least one diagram (C4 context or sequence diagram for the enrollment-to-completion flow); docs/tdd.md describing the red-green-refactor approach with one worked example (e.g., AC-05 quiz auto-grading).

12. Autonomous Fix Loop Evidence. At least one recorded loop where an agent detected a failure (e.g., a duplicate enrollment under retry), reproduced, fixed, validated, and opened a PR. Capture the trace under docs/fix-loops/.

13. Knowledge Deposit Log. docs/knowledge-deposits.md recording recurring mistakes encoded back into rules, hooks, or skills (e.g., a hook that rejects mutations on published course content).

14. Multiple Sprint Cycles. More than one hamess-driven sprint covering: catalog enrollment-lessons + progress quiz engine + instructor workbench.

15. Bonus Agents. Beyond the required additions: janitor, performance-auditor, doc-writer, or other domain-specific or technical agents.

On completion of the capstone, you will have demonstrated the skill the industry is seeking: building a learning management platform with course authoring, enrollment, deterministic quiz grading, and progress tracking without writing the code yourself, while maintaining full control of quality, security, and architecture through AI-native engineering practices.